

LABORATORIUM 6

Responsive Design - implementacja

Zaawansowany Interfejs Użytkownika | 90 minut

Prowadzący	mgr inż. Dominik Aksamit
Uczelnia	Akademia Tarnowska
Kierunek	Informatyka (II stopień)
Laboratorium	6 z 14
Efekty	IN2_U01, IN2_K03, IN2_K04
Technologie	React 18, TypeScript 5, Vite 5, CSS / MUI v6

1. Cele zajęć

Po zakończeniu laboratorium student będzie potrafił:

- **Zastosować podejście mobile-first przy projektowaniu layoutu CSS**
- Implementować responsywne breakpointy (mobile / tablet / desktop) w React + CSS Modules lub MUI v6
- **Tworzyć collapsible navigation (hamburger menu) z obsługą stanu w React useState**
- Używać fluid typography z funkcją clamp() eliminując przeskok między rozmiarami czcionek
- **Budować responsywną siatkę kart z CSS Grid auto-fit bez hardkodowanych szerokości**
- Stosować aspect-ratio i object-fit dla spójnego wyświetlania mediów w kartach
- **Testować responsywność w Chrome DevTools, Firefox RDM i audycie Lighthouse**
- Identyfikować i naprawiać typowe błędy responsywności

INFO

Powiązanie z projektem

Laboratorium kontynuuje aplikację To-Do z Lab 4 i Lab 5. Studenci implementują responsywność na bazie prototypu Figma hi-fi (Lab 3) i komponentów MUI/Tailwind (Lab 5). Kontekst: przygotowanie do Lab 7 - REST API z React Query.

2. Harmonogram zajęć (90 minut)

Czas (min)	Faza	Opis aktywności
0 - 10	Wprowadzenie	Przypomnienie Lab 5 (MUI/Tailwind). Demo problemu: jak wygląda aplikacja na ekranie 375px? Motywacja do mobile-first.

Czas (min)	Faza	Opis aktywności
10 - 25	Live-coding demo	Prowadzący implementuje mobile-first krok po kroku: reset, breakpointy, fluid typography z clamp(). Studenci obserwują i zadają pytania.
25 - 55	Zadanie główne	Studenci implementują responsywność aplikacji To-Do: breakpointy, hamburger menu, siatka kart z aspect-ratio. Prowadzący asystuje.
55 - 65	Debugowanie	Ćwiczenie: 5 błędów responsywności na dostarczonej stronie. Studenci identyfikują i naprawiają samodzielnie.
65 - 78	Nowe właściwości	Omówienie: aspect-ratio + object-fit, brak potrzeby media queries (auto-fit, clamp), container queries, :has(), subgrid, @media print.
78 - 90	Testowanie + Q&A	Lighthouse audit. Prezentacja wyników. Podsumowanie i zapowiedź Lab 7 (REST API + React Query).

3. Zadanie główne - responsywna strona główna aplikacji

Zaimplementuj w aplikacji To-Do pełna responsywność zgodna z projektem Figma z Lab 3. Projekt musi działać poprawnie na wszystkich trzech zakresach ekranu.

3.1. Mobile-first approach

Zasada: style bazowe pisz dla mobile, następnie rozszerzaj dla większych ekranów przez min-width media queries. Nigdy odwrotnie.

```
/* POPRAWNIE - mobile-first */
.task-grid {
  display: grid;
  grid-template-columns: 1fr; /* mobile: 1 kolumna */
  gap: 1rem;
}

@media (min-width: 768px) {
  .task-grid {
    /* TODO: ustaw 2 kolumny dla tabletu (1 linia) */
  }
}

@media (min-width: 1024px) {
  .task-grid {
    /* TODO: ustaw 3 kolumny dla desktop (1 linia) */
  }
}
```

3.2. Breakpointy

Zakres	Szerokosc	Layout siatki	Nawigacja
Mobile	< 768px	1 kolumna (1fr)	Hamburger (zwinięty)
Tablet	768 - 1024px	2 kolumny lub auto-fit	Hamburger lub pozioma

Zakres	Szerokosc	Layout siatki	Nawigacja
Desktop	> 1024px	3+ kolumny	Pełna pozioma nawigacja

3.3. Collapsible navigation - hamburger menu

Nawigacja chowa się na mobilce i rozwija po kliknięciu przycisku hamburger. Użyj useState do zarządzania stanem otwarcia/zamknięcia.

```
// Nav.tsx - szkielet do uzupełnienia
import { useState } from "react";

export const Nav = () => {
  const [isOpen, setIsOpen] = useState(false);

  const handleToggle = () => {
    // TODO: przełącz stan isOpen (1 linia)
  };

  return (
    <nav className="nav">
      <div className="nav__logo">TodoApp</div>

      {/* Przycisk hamburger - widoczny tylko na mobile (CSS) */}
      <button
        className="nav__hamburger"
        onClick={/* TODO: wywołaj handleToggle */}
        aria-label="Otwórz menu"
        aria-expanded={/* TODO: przekaz isOpen */}
      >
        <span /><span /><span />
      </button>

      {/* TODO: klasa warunkowa - dodaj nav__menu--open gdy isOpen === true */}
      {/* Wskazówka: użyj template literal lub classNames() */}
      <ul className={/* TODO */}>
        <li><a href="#">Zadania</a></li>
        <li><a href="#">Kategorie</a></li>
        <li><a href="#">Profil</a></li>
      </ul>
    </nav>
  );
};
```

3.4. Fluid typography z clamp()

clamp(min, preferred, max) definiuje płynny rozmiar czcionki między wartoscia minimalana i maksymalna. Eliminuje skokowe zmiany w media queries.

```
/* _typography.css */
:root {
  /* clamp(min, wartość-skalująca, max) */
  --font-h1: clamp(1.5rem, 4vw, 2.5rem);
  --font-h2: clamp(1.25rem, 3vw, 2rem);

  /* TODO: zdefiniuj --font-body (min: 0.875rem, max: 1.125rem) */
  --font-body: /* TODO (1 linia) */;
}
```

```
/* TODO: zdefiniuj --font-small dla metadanych kart */
--font-small: /* TODO (1 linia) */;
}

h1 { font-size: var(--font-h1); }
h2 { font-size: var(--font-h2); }
body { font-size: var(--font-body); }
```

3.5. Responsywna siatka kart + aspect-ratio

Nowe w tej sekcji: aspect-ratio eliminuje padding-hack dla proporcji elementów. W połączeniu z object-fit zapewnia spójne wyświetlanie obrazów w kartach.

```
/* TaskGrid.css */
.task-grid {
  display: grid;
  /* auto-fit: siatka responsywna BEZ media queries */
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: clamp(0.75rem, 2vw, 1.5rem);
}

.task-card__thumbnail {
  width: 100%;

  /* TODO: ustaw proporcje 16:9 (1 linia) */
  aspect-ratio: /* TODO */;

  /* TODO: wypełnij kontener bez rozciągania obrazu (1 linia) */
  object-fit: /* TODO */;

  /* Dlaczego aspect-ratio > padding-hack? */
  /* STARY: padding-top: 56.25%; height: 0; position: relative; */
  /* NOWY: aspect-ratio: 16 / 9; <- jedna czytelna linia */
}
```

4. Czy zawsze potrzebujemy media queries?

Media queries to jedno z narzędzi - nie jedyne. Warto znać alternatywy, często bardziej eleganckie i lepiej skalujące się w komponentach.

Technika	Kiedy stosować	Przykład
@media queries	Globalne zmiany layoutu strony, nawigacja, liczba kolumn na różnych widokach	@media (min-width: 768px) { ... }
CSS Grid auto-fit	Siatka kart o stałej min. szerokości - layout sam się dostosowuje bez breakpointów	repeat(auto-fit, minmax(280px, 1fr))
clamp() / min() / max()	Płynna typografia i odstępy - zero media queries potrzebnych	font-size: clamp(1rem, 3vw, 2rem)

Technika	Kiedy stosować	Przykład
Container queries	Komponent reaguje na rodzica (nie viewport) - reużywalne niezależnie od miejsca	@container (min-width: 400px) { ... }
Flexbox flex-wrap	Tagi, badges, przyciski - naturalny zawijanie bez breakpointów	display: flex; flex-wrap: wrap; gap: 0.5rem;

TIP

Reguła praktyczna

Używaj media queries do zmian globalnych (sidebar, navbar, liczba kolumn strony). Dla komponentów preferuj auto-fit, clamp() lub container queries. Dobry kod komponentowy powinien być niezależny od tego gdzie jest umieszczony.

5. Ćwiczenie – znajdź i napraw 5 błędów responsywności

Przykładowy plik z błędami (zapisz do pliku .html):

```
<!DOCTYPE html>
<!-- BUG 1: Brak <meta name="viewport"> - przeglądarka mobilna zoomuje stronę x3 -->
<html lang="pl">
<head>
<meta charset="UTF-8">
<!-- <meta name="viewport" content="width=device-width, initial-scale=1.0"> -->
<title>ToDoApp – ćwiczenie</title>
<style>

/* BUG 2: Stałe px zamiast rem – nie skaluje się z preferencjami użytkownika */
body {
margin: 0;
font-family: Arial, sans-serif;
font-size: 12px; /* ✗ powinno być: 1rem lub clamp() */
background: #f2f4f8;
}

h1 { font-size: 20px; } /* ✗ powinno być: clamp(1.25rem, 3vw, 2rem) */
h2 { font-size: 16px; } /* ✗ powinno być: clamp(1rem, 2vw, 1.5rem) */

/* BUG 3: overflow: hidden na body – blokuje scroll na urządzeniach mobilnych */
body {
overflow: hidden; /* ✗ powinno być: overflow-x: hidden (lub usunąć) */
}

/* BUG 4: Nawigacja bez media query – wylewa się poza ekran na mobile */
.nav {
display: flex;
align-items: center;
justify-content: space-between;
background: #0043ff;
padding: 12px 24px;
}

.nav__logo {
color: #fff;
font-weight: bold;
font-size: 18px;
}

.nav__links {
```

```
display: flex; /* ✗ na mobile powinno być: display: none + hamburger */
gap: 24px;
list-style: none;
margin: 0;
padding: 0;
}

.nav__links a {
color: #fff;
text-decoration: none;
white-space: nowrap;
}

/* BUG 5: Za małe touch targets – WCAG 2.5.5 wymaga min. 44x44px */
.btn-icon {
width: 24px; /* ✗ powinno być: min-width: 44px */
height: 24px; /* ✗ powinno być: min-height: 44px */
background: #f8b914;
border: none;
border-radius: 4px;
cursor: pointer;
font-size: 14px;
display: flex;
align-items: center;
justify-content: center;
}

/* --- reszta stylów (poprawna) --- */
.container {
max-width: 960px;
margin: 0 auto;
padding: 24px 16px;
}

.task-grid {
display: grid;
grid-template-columns: repeat(3, 1fr); /* ✗ brak responsywności – przy okazji warto naprawić */
gap: 16px;
margin-top: 24px;
}

.task-card {
background: #fff;
border-radius: 8px;
padding: 16px;
box-shadow: 0 2px 6px rgba(0,0,0,.08);
}

.task-card__title {
font-weight: bold;
margin: 0 0 8px;
}

.task-card__meta {
font-size: 11px; /* ✗ powiązane z BUG 2 */
color: #888;
}

.task-card__actions {
display: flex;
gap: 6px;
margin-top: 12px;
}

</style>
</head>
```

```
<body>

<nav class="nav">
<div class="nav__logo">ToDoApp</div>
<ul class="nav__links">
<li><a href="#">Zadania</a></li>
<li><a href="#">Kategorie</a></li>
<li><a href="#">Ustawienia</a></li>
<li><a href="#">Profil</a></li>
</ul>
</nav>

<div class="container">
<h1>Moje zadania</h1>
<h2>Na dziś</h2>

<div class="task-grid">

<div class="task-card">
<p class="task-card__title">Zaprojektować onboarding</p>
<p class="task-card__meta">Termin: 12.04.2026 · Priorytet: wysoki</p>
<div class="task-card__actions">
<!-- BUG 5 widoczny tutaj – przyciski 24×24px -->
<button class="btn-icon" title="Edytuj"><img alt="edit icon" data-bbox="455 368 475 383"/></button>
<button class="btn-icon" title="Usuń"><img alt="delete icon" data-bbox="455 383 475 398"/></button>
<button class="btn-icon" title="Ukończ"><img alt="check icon" data-bbox="455 398 475 413"/></button>
</div>
</div>

<div class="task-card">
<p class="task-card__title">Code review – komponent Nav</p>
<p class="task-card__meta">Termin: 13.04.2026 · Priorytet: średni</p>
<div class="task-card__actions">
<button class="btn-icon" title="Edytuj"><img alt="edit icon" data-bbox="455 498 475 513"/></button>
<button class="btn-icon" title="Usuń"><img alt="delete icon" data-bbox="455 513 475 528"/></button>
<button class="btn-icon" title="Ukończ"><img alt="check icon" data-bbox="455 528 475 543"/></button>
</div>
</div>

<div class="task-card">
<p class="task-card__title">Napisać testy jednostkowe</p>
<p class="task-card__meta">Termin: 15.04.2026 · Priorytet: niski</p>
<div class="task-card__actions">
<button class="btn-icon" title="Edytuj"><img alt="edit icon" data-bbox="455 628 475 643"/></button>
<button class="btn-icon" title="Usuń"><img alt="delete icon" data-bbox="455 643 475 658"/></button>
<button class="btn-icon" title="Ukończ"><img alt="check icon" data-bbox="455 658 475 673"/></button>
</div>
</div>

</div>
</div>

</body>
</html>
```

Otwórz go w Chrome, przełącz DevTools na iPhone 12 Pro (390px) i znajdź wszystkie problemy w ciągu 10 minut.

#	Błąd	Lokalizacja / kod	Skutek dla użytkownika
1	Brak meta viewport	<meta name="viewport"> nieobecny w <head>	Strona nieużywalna na mobile - przeglądarka zoomuje x3
2	Stałe px zamiast rem	font-size: 12px na body, nie skaluje się	Tekst nieczytelny na małych ekranach
3	overflow: hidden na body	body { overflow: hidden } blokuje scroll mobile	Użytkownik nie może przewijać strony
4	Brak media query na nav	.nav { display: flex } bez zwijania na mobile	Nawigacja wylewa się poza ekran
5	Za małe touch targets	Przyciski 24x24px, wymagane >= 44x44px (WCAG 2.5.5)	Nieemożliwe do kliknięcia palcem

ZADANIE**Instrukcja**

Dla każdego błędu: (1) opisz słownie co widzisz, (2) znajdź przyczynę w kodzie HTML/CSS, (3) napisz poprawiony fragment CSS. Porównaj wyniki z sąsiadem - czy znaleźliście te same błędy?

6. Nowe właściwości CSS

6.1. aspect-ratio + object-fit

aspect-ratio definiuje proporcje elementu. **object-fit** kontroluje skalowanie mediów wewnątrz kontenera. Razem zastępują padding-hack i pozwalają na spójne karty bez JavaScript.

```
/* aspect-ratio: proporcje elementu */
.video-embed { aspect-ratio: 16 / 9; } /* film */
.avatar { aspect-ratio: 1; } /* kwadrat / kółko */
.product-card { aspect-ratio: 3 / 4; } /* pionowy kafelek */

/* object-fit: jak obraz wypełnia kontener */
.thumbnail {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover; /* przytnij, zachowaj proporcje - najczęściej */
  /* object-fit: contain; <- cały obraz widoczny, letterbox */
  /* object-fit: fill; <- rozciągnij (unikaj!) */
  object-position: center top; /* punkt kadrowania */
}

/* Stary padding-hack (do unikania od 2021):
padding-top: 56.25%; height: 0; position: relative; overflow: hidden; */
```

6.2. Container Queries (@container)

Container queries pozwalają komponentowi reagować na rozmiar swojego rodzica - nie viewportu. Umożliwia tworzenie naprawdę reużywanych komponentów niezależnych od kontekstu.

VS

Media query vs. Container query

Media query: "jesli okno $\geq$ 768px zmien layout" - komponent musi znać swoje miejsce na stronie. Container query: "jeśli mój kontener $\geq$ 400px zmień layout" - komponent jest autonomiczny, można go użyć w sidebar, main lub modalu.

```
/* 1. Oznacz rodzica jako kontener */
.card-wrapper { container-type: inline-size; }

/* 2. Styl bazowy (mobile-first) */
.task-card {
  display: flex;
  flex-direction: column;
}

/* 3. Container query - reaguje na rodzica, nie na viewport */
@container (min-width: 400px) {
  .task-card {
    flex-direction: row;
    align-items: center;
  }
}

/* Ten sam komponent w wąskim sidebarze = kolumna */
/* Ten sam komponent w szerokiej sekcji main = wiersz */
/* Zero dodatkowych media queries! */
```

6.3. Selektor :has()

:has() to tzw. "parent selector" - element może być stylizowany na podstawie swoich potomków. Wsparcie: Chrome 105+, Firefox 121+, Safari 15.4+.

```
/* Karta z obrazkiem otrzymuje dodatkowy rząd gridu */
.task-card:has(img) {
  grid-template-rows: auto 1fr auto;
}

/* Czerwona etykieta gdy input niepoprawny */
.form-group:has(input:invalid) label {
  color: red;
}

/* Zablokuj scroll body gdy menu otwarte */
body:has(.nav__menu--open) {
  overflow: hidden;
}

/* Layout dostosowuje się do obecności sidebara */
.layout:has(.sidebar:not([hidden])) .main {
  margin-left: var(--sidebar-width);
}
```

6.4. CSS Subgrid

Subgrid pozwala elementom potomnym uczestniczyć w siatce rodzica. Rozwiązuje klasyczny problem: karty o różnej długości tekstu mają niespójne wewnętrzne wyrównanie.

```
.task-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  grid-template-rows: auto;
}

.task-card {
  display: grid;
  grid-row: span 3;
  grid-template-rows: subgrid; /* dziedzicz rzędy z rodzica */
}

/* Teraz tytuły, opisy i footery są wyrównane między kartami */
.card__title { grid-row: 1; }
.card__body { grid-row: 2; }
.card__footer { grid-row: 3; align-self: end; }

/* Wsparcie: Chrome 117+, Firefox 71+, Safari 16+ */
```

6.5. @media print

@media print to często pomijany aspekt responsywności. Pozwala zdefiniować style dla wydruku lub eksportu PDF. Szczególnie przydatny przy listach zadań, raportach, fakturach.

```
@media print {
  /* Ukryj elementy interfejsu */
  .nav,
  .btn-add-task,
  .sidebar {
    display: none !important;
  }

  body {
    background: white;
    color: black;
    font-size: 12pt;
  }

  /* Nie łam kart w środku strony */
  .task-card {
    break-inside: avoid;
    border: 1px solid #ccc;
    margin-bottom: 1rem;
  }

  /* Klasy pomocnicze */
  .no-print { display: none; }
  .print-only { display: block; } /* domyślnie hidden */
}
```

TIP

Testowanie @media print w DevTools

Chrome DevTools > ikona "..." > More tools > Rendering > Emulate CSS media type > print. Możesz testować styl wydruku bez drukarki. Sprawdź też: Ctrl+P > Save as PDF - tak widzą Twoją aplikację użytkownicy drukujący listy zadań.

7. Instrukcja testowania responsywności

7.1. Chrome DevTools - Device Mode

1. Otwórz DevTools: F12 lub Ctrl+Shift+I
2. Kliknij ikonę "Toggle Device Toolbar" (Ctrl+Shift+M)
3. Wybierz urządzenie (np. iPhone 12 Pro - 390px) lub ustaw wymiary ręcznie
4. Przeciągnij uchwyt - sprawdź czy layout płynnie przechodzi przez breakpointy
5. Włącz throttling: Network > Fast 4G (symuluj realne warunki mobilne)

TIP**Ukryta funkcja Chrome**

W trybie responsywnym: "..." > Show media queries - kolorowe paski na linijce pokazują Twoje breakpointy. Kliknięcie paska przełącza do danego breakpointa.

7.2. Firefox Responsive Design Mode

6. Otwórz: F12 > ikona telefonu lub Ctrl+Shift+M
7. Włącz Touch Simulation - kliknięcia zachowują się jak dotyk (przydatne do testów hamburger menu)
8. Zapisz własne wymiary urządzeń do listy predefiniowanych

7.3. Lighthouse Audit

9. DevTools > zakładka Lighthouse
10. Zaznacz: Performance, Accessibility, Best Practices
11. Wybierz urządzenie: Mobile
12. Kliknij "Analyze page load" - oczekiwany wynik: Accessibility >= 90
13. Rozwiń sekcję Accessibility - zwróć uwagę na: touch targets, kontrast, atrybuty ARIA nawigacji

7.4. Testowanie na realnym urządzeniu

14. W vite.config.ts dodaj: server: { host: true }
15. npm run dev - terminal wyświetli adres LAN (np. http://192.168.1.10:5173)
16. Wpisz adres w przeglądarce mobilnej (ta sama sieć WiFi)
17. Sprawdź: scroll, touch targets, hamburger menu, orientacja pozioma (landscape)

8. Kryteria oceny

Ocena wystawiana jest na podstawie oddanego kodu.

Kryteria zaliczenia	Status
Wszystkie kryteria 5.0 spełnione. Container queries LUB CSS subgrid LUB @media print w projekcie. Lighthouse Accessibility >= 95.	ZALICZONE
Pełna responsywność (3 zakresy). Hamburger menu. clamp() dla typografii. aspect-ratio na kartach. Lighthouse Performance >= 90, Accessibility >= 90.	ZALICZONE
Jak 4.0 oraz: aspect-ratio poprawnie zastosowany. Siatka bez hardkodowanych szerokości. Testowanie w DevTools udokumentowane.	ZALICZONE
Wszystkie breakpointy zaimplementowane. Hamburger menu działa. clamp() na min. 2 elementach. Grid kart responsywny.	ZALICZONE
Mobile-first zachowane. Co najmniej 2 breakpointy. Nawigacja chowa się na mobile. Widoczne drobne błędy na tablet/desktop.	ZALICZONE
Podstawowy layout mobilny działa. Min. 1 media query. Hamburger menu obecne (może być niekompletne). Brak breakpointa tablet.	ZALICZONE
Brak mobile-first. Brak działających breakpointów. Aplikacja nieużywalna na 375px. Zadanie niezakończone lub nieoddane.	NIEZALICZONE

Zadanie dodatkowe

Wybierz i zaimplementuj jedno z poniższych rozszerzeń:

- Container queries: co najmniej 1 komponent reaguje na rozmiar kontenera zamiast viewportu
- CSS Subgrid: karty w siatce mają wyrównane sekcje (tytuł / opis / footer) bez względu na długość tekstu
- @media print: lista zadań poprawnie stylizowana przy eksporcie PDF - ukryta nawigacja, poprawne page-break

9. Materiały i literatura

Dokumentacja online

- MDN - Responsive Design: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
- MDN - Container Queries: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_queries
- MDN - aspect-ratio: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/aspect-ratio>
- Google - Responsive Web Design Basics: <https://web.dev/articles/responsive-web-design-basics>
- Can I Use (wsparcie przeglądarek): <https://caniuse.com>